

1 chapter10: 随机算法

1.1 导论

1. 与确定性算法相比，随机算法所需运行时间或空间通常较小；
2. 随机算法易于理解和实现。

1. Las Vegas 算法¹：总是给出正确的解，或者无解

通常需要分析算法的平均运行时间，运行时间本身是一个随机变量。

期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为有效的拉斯维加斯算法，也称为 **ZPP 类算法** (zero-error probabilistic polynomial time) 算法，即零错误概率多项式时间随机算法。

2. Monte Carlo 算法：总是给出解，偶尔可能是错误解，可以通过多次运行原算法，且满足每次运行时随机选择都相互独立，使产生错误解的概率减到任意小。

通常需要分析算法的出错概率，总是在多项式时间内运行且出错概率不超过 **1/3** 的随机算法称为有效的 * 蒙特卡罗型随机算法，也称为 **BPP 类算法** (bounded-error probabilistic polynomial time)，即错误概率有限的多项式时间随机算法。

错误类型：

- 弃真型错误：在求解一个判定问题时把本应该接受的输入误判为拒绝。
- 取伪型错误：就是把本应该拒绝的输入误判为接受。

根据是否同时出现两类错误，蒙特卡罗型算法又分为：

- (a) **单侧错误的蒙特卡罗型算法**：在所有输入上，要么犯弃真的错误，要么犯取伪的错误，不同时出现两种不同的错误。
 - 有效的弃真型单侧错误随机算法称为 **RP 类算法** (Randomized Polynomial-time)
 - 有效的取伪型单侧错误随机算法称为 **coRP 类算法** (Complementary Randomized Polynomial-time, RP 的补类)
- (b) **双侧错误的蒙特卡罗型算法**：在所有输入上可以同时出现两种不同的错误。

对于随机算法 A, 其在大小为 n 的一个实例 I 上的运行时间可能因不同次执行而异。因此，更自然的性能度量是 **A 在实例 I 上的期望运行时间**，也即算法 A 反复求解实例 I 所花费的平均时间。

- 对于确定算法，求解期望运行时间时，概率分布定义在**输入上**
- 对于随机算法，求解期望运行时间时，概率分布定义在**算法上**

期望时间：对于大小为 n 的特定输入 I, 算法的期望时间就是在随机数的不同序列上的时间的平均：

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

¹Las Vegas 和 Monte Carlo 不是一种算法的名称，而是对一类随机算法的特性的概括。不太可能为 **NP 完全问题** 设计出多项式时间的随机算法

最坏情况下的期望时间:

$$W_ET(A) = \max_{\text{input } I \text{ with size } n} (\text{random sequence } R \text{probability}(R) * \text{time}(A, I, R))$$

1.2 随机算法举例

这是一个很不错的讲解栏目: <https://www.luogu.com.cn/article/wegx023s>

1.2.1 随机快速排序

其实是在快速排序的基础上, 增加了随机选取的步骤。

```
Input : An array  $A[1 \dots n]$  of  $n$  elements
Output: The elements in  $A$  sorted in nondecreasing order

1 quicksort (A,low,high)
2 if low < high then
3   | SPLIT(A [low...high ], w)
4   | // w is the new position of A [low ]
5   | quicksort (A,low, w-1)
6   | quicksort (A, w +1, high)
7 end
```

Algorithm 1: Quicksort

```
Input : An array  $A$  of  $n$  elements
Output: The elements in  $A$  sorted in nondecreasing order

1 rquicksort (A,low,high)
2 if low < high then
3   |  $v \leftarrow \text{random}(\text{low}, \text{high})$  interchange  $A[\text{low}]$  and  $A[v]$ 
4   | SPLIT(A [low...high ], w, v)
5   | rquicksort (A,low, w-1)
6   | rquicksort (A,w +1, high)
7 end
```

Algorithm 2: Randomized Quicksort

该算法在最坏情况的运行时间仍然是 $\Theta(n^2)$, 但这个最坏情况不是由于输入形式, 而是由于随机数生成器每次非常不幸地选择了不好的划分元素, 这种情况不大可能发生。算法的期望运行时间为 $\Theta(n \log n)$, 是一个 ZPP 类算法

Input : An array A of n elements, a number k such that $1 \leq k \leq n$

Output: The k th smallest element in A

```
1 select(A, 1, n, k)
2 p ← high- low + 1
3 if p < 44 then
4   | sort A and return A [k]
5 end
6 let q = ⌊p/5⌋. Divide A into q groups of 5 elements each. If 5 doesn't divide p, then discard the
   remaining elements.
7 // Although some elements are discarded here, the subsequent grouping still
   takes them into account, so it will not affect the answer.
8 Sort each of the q groups individually and extract its median element. Let the set of medians be
   M
9 mm ← select (M, 1, q, ⌊q/2⌋) // median of medians
10 Partition A into three arrays:
11 A1 ← {A|A < mm}, A2 ← {A|A = mm}, A3 ← {A|A > mm}
12 if |A1| ≥ k then
13   | return select (A1, 1, |A1|, k)
14 else
15   | if k ≤ |A1| + |A2| then
16     |   return mm
17   | else
18     |   return select (A3, 1, |A3|, k - |A1| - |A2|)
19   | end
20 end
```

Algorithm 3: SELECT algorithm

1.2.2 随机选择

在最坏情况下，随机数生成器在第 j 次迭代中选择第 j 小的元素，因此只丢弃一个元素。由于将 A 划分为 A_1 、 A_2 和 A_3 所需的比较次数正好为 n ，因此算法在最坏情况下进行的元素比较总数为

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

设 $C(n)$ 为对输入数组有 n 个元素时进行元素比较运算的期望次数。由于随机选择的 v 可以等概率假设为 $1, 2, \dots, n$ 中的任何一个，因此根据 $v < k$ 或 $v > k$ ，有两种情况需要考虑。

- $v < k$: 剩余元素的数量为 $n - v$ (v 右侧的元素数量为 $n - v$)
- $v > k$: 剩余元素的数量为 $v - 1$ (v 左侧的元素数量为 $v - 1$)

因而有：

$$C(n) = n + \frac{1}{n} \sum_{v=1}^n \begin{cases} C(v-1) & \text{if } v < k \\ C(n-v) & \text{if } v > k \end{cases}$$

Input: An array A of n elements, a number k such that $1 \leq k \leq n$

Output: The k th smallest element in A

```
1 rselect( $A, 1, n, k$ )
2  $v \leftarrow \text{random}(\text{low}, \text{high})$ 
3  $x \leftarrow A[v]$ 
4 Partition  $A[\text{low}...\text{high}]$  into three arrays:  $A_1 \leftarrow \{A|A < x\}$ ,  $A_2 \leftarrow \{A|A = x\}$ ,  $A_3 \leftarrow \{A|A > x\}$ 
5 if  $|A_1| \geq k$  then
6   return rselect( $A_1, 1, |A_1|, k$ )
7 else
8   if  $k \leq |A_1| + |A_2|$  then
9     return  $x$ 
10  else
11    return rselect( $A_3, 1, |A_3|, k - |A_1| - |A_2|$ )
12  end
13 end
```

Algorithm 4: Randomized SELECT

由数学归纳法, 假设 $C(m) \leq 4m$ 对所有 $m < n$ 成立, 下证 $C(n) \leq 4n$:

$$\begin{aligned} C(n) &\leq n + \frac{1}{n} \sum_{v=1}^n 4 \cdot \max(n-v, v-1) \\ &\leq n + \frac{8}{n} \cdot \sum_{v=1}^{\frac{n}{2}} (n-v) \\ &\leq n + \frac{8}{n} \cdot \frac{3n^2}{8} \\ &= 4n \end{aligned}$$

又由于每个元素至少被检查一次, 因此 $C(n) \geq n$, 因此算法的期望运行时间是 $\Theta(n)$, 是 ZPP 类算法。

1.2.3 多选问题

设有 n 个元素的数组 A , 并且设 $K[1..r]$ ($1 \leq r \leq n$) 是一个由 1 到 n 之间的 r 个正数构成的有序数组, 即一个排序数组。

多选问题就是对于所有 i , $1 \leq i \leq r$, 选择 A 中的第 $K[i]$ 小的元素。假定 $A[1..n]$ 中所有元素各不相同。如果 $r = 1$, 该问题即为选择问题。如果 $r = n$, 该问题即为排列问题。

设 $A[1..n] = [a_1, a_2, \dots, a_n]$; $K[1..r] = [k_1, k_2, \dots, k_r]$, 设中间排位为 $k = k_{\frac{r}{2}}$, 可使用 SELECT 算法发现第 k 小的元素 a 。接着根据 a 把 A 划分成两个子序列 A_1 和 A_2 , 其中 A_1 中元素个数大于等于 a , A_2 中元素个数小于等于 a 。令 $K_1 = [k_1, k_2, \dots, k_{\frac{r}{2}}]$, $K_2 = [k_{\frac{r}{2}+1}, k_{\frac{r}{2}+2}, \dots, k_r]$ 。递归调用, 一个关于 A_1 和 K_1 的递归调用, 一个关于 A_2 和 K_2 的递归调用。

Multiselect 算法的时间复杂度是 $\Theta(n \log r)$ 。这是因为调用 SELECT 算法的复杂度是 $\Theta(n)$, 所以有

Input: n 元序列 A , 具有 r 个排位的有序序列 K

Output: A 中第 k_i 小的元素

```
1 multiselect( $A, K$ )
2  $r \leftarrow |K|$ 
3 if  $r > 0$  then
4    $k = k_{\frac{r}{2}}$ 
5    $a \leftarrow \text{SELECT}(A, k)$ 
6    $\text{Ans}[\frac{r}{2}] \leftarrow a$ 
7    $A_1 = [a_i | a_i < a], A_2 = [a_i | a_i > a]$ 
8    $K_1 \leftarrow [k_1, k_2, \dots, k_{\frac{r}{2}}], K_2 \leftarrow [k_{\frac{r}{2}+1} - k, \dots, k_r - k]$ 
9   // 特别注意这里需要减去  $k$ 
10  multiselect( $A_1, K_1$ )
11  multiselect( $A_2, K_2$ )
12  return  $\text{Ans}$ 
13 end
```

Algorithm 5: Multiselect

递推式

$$T(n, r) = \Theta(n) + T(n_1, \frac{r}{2}) + T(n_2, \frac{r}{2}), n_1 + n_2 \leq n$$

递归深度是 $\log r$, 所以时间复杂度是 $\Theta(n \log r)$.

将多选问题要寻找的元素称为靶标。随机一致地挑选一个元素 a (而不是通过 SELECT 精心选择), 将数组 A 关于 a 分割成两部分: 一部分元素小于 a , 另一部分元素大于 a 处理逻辑:

如果两部分均含有靶标, 则递归调用算法继续求解; 否则, 丢弃不含有靶标的部分, 对含有靶标部分继续求解

该算法以概率 $1 - O(\frac{1}{n})$ 的时间复杂度为 $O(n \log r)$. r 个靶标的分配可以看作 r 次独立伯努利试验, 其分布近似二项分布 $B(r, 1/2)$ 。根据 Hoeffding 不等式, 靶标分配的偏离概率满足:

$$P\left(\left|\text{左侧靶标数} - \frac{r}{2}\right| \geq \epsilon r\right) \leq 2 \exp(-2\epsilon^2 r)$$

当 $r \geq c \log n$ 时, 此概率上界为 $O(1/n)$ 。

1.2.4 随机取样

首先将所有 n 个元素标记为未选中。接下来, 重复以下步骤, 直到选择了 m 个元素:

1. 生成一个介于 1 和 n 之间的随机数 r
2. 如果 r 被标记为未选中, 则将其标记为选中并将其添加到样本中

算法的期望运行时间 $T(n) = \Theta(n)$ 如果 $m > \frac{n}{2}$, 那么应该考虑反面, 改为随机选择 $n - m$ 个整数丢弃, 将剩下的整数添加到样本中。

设第 k 次选择时已选中 k 个元素, 则:

- 成功选中新元素的概率 $p_k = \frac{n-k}{n}$

- 失败（重复选中）的概率 $q_k = 1 - p_k = \frac{k}{n}$

根据几何分布，第 $k+1$ 个新元素的期望尝试次数为：

$$E_k = \frac{1}{p_k} = \frac{n}{n-k}$$

每次尝试的时间成本为 $\Theta(1)$ （仅需检查布尔数组 S ）。

$$\begin{aligned} T(n) &= \sum_{k=0}^{m-1} E_k \cdot \Theta(1) \\ &= \sum_{k=0}^{m-1} \frac{n}{n-k} \\ &= n \left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{n-m+1} \right) \\ &= n \cdot (H_n - H_{n-m}) \quad (\text{调和数展开}) \end{aligned}$$

利用调和数近似 $H_n \approx \ln n + \gamma$ ：

$$T(n) \approx n (\ln n - \ln(n-m)) = n \ln \left(\frac{n}{n-m} \right)$$

- 当 $m = O(n)$ 时（如 $m = \sqrt{n}$ ）： $T(n) \approx n \cdot \frac{m}{n} = \Theta(m)(\ln x \approx x - 1)$
- 当 $m = \Theta(n)$ 时（如 $m = n/2$ ）： $T(n) = \Theta(n)$

因此对于任意 $m < n$ ，最坏情况下 $T(n) = \Theta(n)$ 。

1.2.5 素数测试

易知除 2, 3 以外的所有素数都是 $6k \pm 1$ 的形式，因此可以用 $6k \pm 1 \leq \sqrt{n}$ 来对 n 是否是素数进行测试。

1. 随机抽取一个数 a ($1 \leq a < n$)；
2. 检查等式：验证涉及 a 和 n 的特定数论等式（与所选测试方法对应）。
若等式不成立，则：

- n 是合数（Composite）
- a 称为合数的证据（Witness）
- 测试终止

3. 重复迭代：返回步骤 1，直到达到预设置信度。

现在我们先介绍幂运算：如果每次乘法花费 1 个时间单元，算法的运行时间是 $\Theta(\log n)$ 。但如果任意大的整数，两数相乘的时间是 $O(\log^2 n)$ ，则算法的运行时间是 $O(\log^3 n)$ 。

利用费马定理：如果 n 是素数，那么对于所有的 $a \not\equiv 0 \pmod{n}$ ，都有 $a^{n-1} \equiv 1 \pmod{n}$ ，如果通过 a 的测试，则成 n 是基 2 的伪素数，例如 341。而对于小于 100000 的数中，只有 78 个数计算出错，这是因为费马定理的逆并不正确。一类被称为卡迈克尔数的合数 n ，它对于所有相对于 n 互素的正整数 a 均满足费马定理，他们在小于 $1e8$ 的范围内只有 255 个。

Input: Positive integer n , positive integer a

Output: Positive integer a^n

```
1 Let the binary digits of  $n$  be  $b_k, b_{k-1}, \dots, b_0$ 
2  $c \leftarrow 1$ 
3 for  $j \leftarrow k$  downto 0 do
4    $c \leftarrow c \cdot 2$ 
5   if  $b_j$  is odd then
6      $c \leftarrow c \cdot a$ 
7   end
8 end
9 return  $c$ 
```

Algorithm 6: EXP

引理：如果合数 n 不是一个卡迈克尔数，那么算法（随机选择 a 并验证费马定理）将检测出 n 为合数的概率至少是 $\frac{1}{2}$ 。

若 n 是合数但非卡迈克尔数，则存在至少一个 $b \perp n$ 使得 $b^{n-1} \not\equiv 1 \pmod{n}$ ，则定义集合

$$W = \{a \in \{1, 2, \dots, n-1\} | a^{n-1} \not\equiv 1 \pmod{n}\}$$

而对于 W 在整数集 Z_n^* 上的补集 G ，易知 G 是 Z_n^* 的一个乘法子群，那么由拉格朗日定理知 $|G|$ 整除 $|Z_n^*| = \phi(n)$ ，所以 $|G| \leq \frac{1}{2}\phi(n)$ ，所以 $|W| \geq \frac{1}{2}\phi(n)$ ，引理得证。

再考虑 Miller-Rabin 算法，设 n 为奇素数，则存在 q, m 使得：

$$n-1 = 2^q m (q \leq 1)$$

构造序列：

$$a^m \pmod{n}, a^{2m} \pmod{n}, a^{4m} \pmod{n}, \dots, a^{2^{q-1}m} \pmod{n}$$

由费马定理知，最后一项 $a^{n-1} \equiv 1 \pmod{n}$ 。而如果 $x^2 \equiv 1 \pmod{n}$ 的根是 $x = \pm 1$ 称为平凡根，其他根是不平凡的，如果存在非平凡根，那么一定是合数。

所以检测该序列中某项是 1，那么前一项必须是 1 或者 $n-1$ 否则 n 是合数。

每次测试出错（把合数判定为素数）概率至多是 $\frac{1}{2}$ ，这样我们重复运行 k 次，就可以将出错的概率降低到 2^{-k} ，那么取 $k = \lceil \log n \rceil$ ，失败的概率不超过 $2^{-\lceil \log n \rceil} \leq \frac{1}{n}$ ，而这就是 **Miller-Rabin 算法**。

1.2.6 验证串的相等性

对于一个比特串 w ，设 $I(w)$ 是 w 表示的一个整数，那么可以选一个素数 p ，这样得到一个指纹函数：

$$I_p(x) = I(x) \pmod{p}$$

对于 $I_p(x) = I_p(y)$ ，而 $x \neq y$ 的现象，我们称之为**假匹配**。每次验证时重新选择 p ，并重复选择 p ，可以增强 $x = y$ 的可信度。

我们知道，设 $\pi(n)$ 是小于 n 的不同素数的个数，那么 $\pi(n)$ 趋近于 $\frac{n}{\ln n}$ 。对于两个长度为 n 的串 x, y ，如果发生假匹配，那么 $p | (I(x) - I(y))$ ，所以其概率为：

$$\frac{|\{p \leq M | p \text{ is prime and } p | (I(x) - I(y))\}|}{\pi(M)} \leq \frac{\pi(n)}{\pi(M)}$$

Input: n

Output: n 是合数或者素数

```
1  $q \leftarrow 0, m \leftarrow n - 1$ 
2 repeat
3    $m \leftarrow \frac{m}{2}$ 
4    $q \leftarrow q + 1$ 
5 until  $m$  是奇数;
6  $a \leftarrow \text{Random}(2, n - 1)$ 
7  $x_0 \leftarrow \text{EXPMOD}(a, m, n)$  //  $x_0 = a^m \pmod n, O(\log^3 n)$ 
8 for  $i \leftarrow 1$  to  $q$  do
9    $x_i \leftarrow x_{i-1}^2 \pmod n$  //  $O(\log^2 n)$ 
10  if  $x_i = 1$  and  $x_{i-1} \neq n - 1$  and  $x_{i-1} \neq 1$  then
11    return 合数 // 100% correct
12  end
13 end
14 if  $x_q \neq 1$  then
15   return 合数 // 100% correct
16 end
17 return 素数 // not 100% correct
```

Algorithm 7: 单次的 Miller-Rabin 素数判定算法

关键观察：假匹配发生时，素数 p 必须满足 $p \mid (I(x) - I(y))$ 。由于 x 和 y 是长度为 n 的比特串， $I(x)$ 和 $I(y)$ 的绝对值不超过 $2^n - 1$ ，因此

$$|I(x) - I(y)| \leq 2^n - 1.$$

(这个结论需要在 $n > 11$ 后成立，这是由切比雪夫函数完成估计的，最终 n 以内的素数的乘积的增长速率约为 $e^{0.9n}$ ，是高于 $2^n - 1$ 这个速率的，所以该估计是正确的)

而如果取 $M = n^2$ ，重复执行 k 次算法，这样最后概率最大是 $(\frac{2}{n})^k$ 。

1.2.7 模式匹配

确定模式串是否出现在文本串中 (假设都是 0,1 串)。利用随机算法比较模式指纹 $I_p(Y)$ 和文本块 $I_p(X(j))$ ，让文本块的指纹要易于计算，最好还可以通过 $X(j)$ 计算出 $X(j+1)$

仿照上一题思想，我们把这里的指纹就假定为选定的串对应的数的大小，那么有：

$$I_p(X(j+1)) = (2I_p(X(j)) - 2^m x_j + x_{j+m}) \pmod p$$

易知该算法的时间复杂度是 $O(m + n)$

1.2.8 货郎担问题

假设有一个旅行商人要拜访 n 个城市，路径选择需满足以下限制：

Input: 长度为 n 的文本串 X , 长度为 m 的模式串 Y
Output: 模式串 Y 在文本串 X 中的第一个匹配位置, 未匹配返回 0

```

1 在小于  $M$  的素数中随机选择素数  $p$ 
2  $j \leftarrow 1$ 
3 计算  $I_p(Y), I_p(X_j)$ 
4 while  $j \leq n - m + 1$  do
5     if  $I_p(Y) = I_p(X_j)$  then
6         return  $j$ 
7     end
8      $I_p(X_{j+1}) \leftarrow f(I_p(X_j), p)$ 
9      $j \leftarrow j + 1$ 
10 end
11 return 0

```

Algorithm 8: Rabin-Karp 算法

1. 每个城市只能拜访一次;
2. 最终必须回到出发城市。

优化目标: 选择总路程最短的路径

Input: 城市数 n , 每两城市间距离矩阵 D
Output: 最优路径 $bestPath$, 路径总长 $bestDistance$

```

1  $bestPath \leftarrow []$  // 初始化最优路径为空
2  $bestDistance \leftarrow \infty$  // 初始化最短距离为无穷大
3  $path \leftarrow$  随机排列( $1, 2, \dots, n$ ) // 生成初始随机路径
4  $distance \leftarrow$  计算路径总长度( $path, D$ ) // 计算初始路径长度
5 for  $j \leftarrow 1$  to  $iteration$  do
6     if  $distance < bestDistance$  then
7          $bestPath \leftarrow path$  // 更新最优路径
8          $bestDistance \leftarrow distance$  // 更新最短距离
9     end
10     $path \leftarrow$  生成新随机路径() // 重新采样路径
11     $distance \leftarrow$  计算路径总长度( $path, D$ ) // 重新计算长度
12 end
13 return  $bestPath, bestDistance$  // 返回找到的最优解

```

Algorithm 9: 随机采样求解 TSP 算法 (RandomTSP)

1.3 随机游走算法

常用有限马尔科夫链来分析随机游走算法。

马尔可夫特性: 随机过程在给定 t 时刻的状态为 X_t , 该过程的后续状态及其出现的概率与 t 之前的

状态无关。也就是说，过程当前的状态包括了过程所有的历史信息，该过程的进一步发展完全由当前状态所决定，与当前状态之前的历史无关，这种性质也称为**无后效性**或**无记忆性**。

马尔科夫不等式：设 X 为只取非负的随机变量，数学期望 $E(X) = \mu$ ，对于任意 $\varepsilon > 0$, $P(X \geq \varepsilon) \leq \frac{\mu}{\varepsilon}$

1. **Integral method** (for continuous case):

$$E(X) = \int_0^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} xf(x)dx \geq \int_{\varepsilon}^{\infty} \varepsilon f(x)dx \geq \varepsilon P(X \geq \varepsilon)$$

2. **Indicator method** (general): Define $Y = \varepsilon \cdot \mathbf{1}_{\{X \geq \varepsilon\}}$, then:

$$E(Y) = \varepsilon P(X \geq \varepsilon) \leq E(X)$$

设一个离散的随机序列 $\{X_0, X_1, \dots\}$ ，其中每个 X_i 都从有限集中取值，若它满足

$$P(X_{i+1} = x_{i+1} | X_i = x_i, X_{i-1} = x_{i-1}, \dots, X_0 = x_0) = P(X_{i+1} = x_{i+1} | X_i = x_i) = p_{x_i, x_{i+1}}$$

这称之为**有限马尔科夫链**

对于有限马氏链，不妨设 X_t 取值的状态空间为 $\{0, 1, \dots, n\}$ ，于是 $p_{i,j}$ （从状态 i 转移到状态 j 的概率）可组成一个 $n+1$ 阶方阵

$$P = [p_{i,j}],$$

称为**一步转移矩阵**，每一行元素之和等于 1。

设 $p_i(t)$ 表示在时刻 t 处于状态 i 的概率，则

$$p_i(t) = p_0(t-1)p_{0i} + p_1(t-1)p_{1i} + \dots + p_{n-1}(t-1)p_{(n-1)i} + p_n(t-1)p_{ni}$$

设 $\mathbf{p}(t)$ 表示向量 $(p_0(t), p_1(t), \dots, p_n(t))$ ，有

$$\mathbf{p}(t) = \mathbf{p}(t-1)P$$

对于任意 m ，定义 m 步转移矩阵

$$P^{(m)} = [p_{i,j}^{(m)}],$$

其中

$$p_{i,j}^{(m)} = P(X_{t+m} = j | X_t = i).$$

因此，

$$P^{(m)} = P^m, \mathbf{p}(t+m) = \mathbf{p}(t)P^m.$$

1.4 二元布尔可满足问题

Input: 整数 n (变元数量), 整数 m (子句数量), 合取范式 F (每个子句至多两个文字)

Output: 可满足赋值, 或宣布不可满足

```

1 随机初始化赋值  $\sigma$ : 对每个变元  $x_i$  随机赋 True 或 False;
2 for  $k \leftarrow 1$  to  $2mn^2$  do
3   if  $\sigma$  满足  $F$  then
4     return  $\sigma$ ; // 找到可满足解
5   end
6   随机选择  $F$  中一个不满足的子句  $C$ ;
7   从  $C$  中随机选择一个文字  $l$ ;
8   翻转  $l$  对应变元的赋值; // 将 True 改为 False 或反之
9 end
10 return "不可满足"; // 超过最大迭代次数未找到解

```

Algorithm 10: 2-SAT 随机局部搜索算法

定理: 若输入公式是不可满足的, 则上述随机游走算法正确宣布不可满足; 若输入公式是可满足的, 则上述算法以 $1 - \frac{1}{2^m}$ 概率找到可满足赋值。

定义马尔可夫链 $\{Y_0, Y_1, Y_2, \dots\}$ 如下:

$$\begin{aligned}
 Y_0 &= X_0 \\
 P(Y_{t+1} = 1 | Y_t = 0) &= 1 \\
 P(Y_{t+1} = j + 1 | Y_t = j) &= \frac{1}{2} \\
 P(Y_{t+1} = j - 1 | Y_t = j) &= \frac{1}{2} \quad (j > 0)
 \end{aligned}$$

显然有:

$$\begin{aligned}
 P(Y_{t+1} = j + 1 | Y_t = j) &= \frac{1}{2} \geq P(X_{t+1} = j + 1 | X_t = j) \\
 P(Y_{t+1} = j - 1 | Y_t = j) &= \frac{1}{2} \geq P(X_{t+1} = j - 1 | X_t = j)
 \end{aligned}$$

从任意状态出发, Y_t 比 X_t 花费更长的期望时间进入状态 n 。因此 Y_t 进入状态 n 的时间作为算法期望运行时间的上界。

设 h_j 表示从状态 j 到状态 n 的期望运行时间, 则有:

$$\begin{aligned}
 h_n &= 0 \\
 h_0 &= h_1 + 1 \\
 h_j &= \frac{h_{j+1} + 1}{2} + \frac{h_{j-1} + 1}{2}, \quad 0 < j < n
 \end{aligned}$$

由归纳法可证, 对于所有 j , 有:

$$h_j \leq n^2 - j^2 \leq n^2$$

因此有:

$$E[X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间}] \leq E[Y \text{ 从 } Y_0 \text{ 到达 } n \text{ 的时间}] \leq n^2$$

取 $\varepsilon = 2n^2$, 根据马尔可夫不等式, 则有:

$$P(X \text{ 从 } X_0 \text{ 到达 } n \text{ 的时间} \geq 2n^2) \leq \frac{n^2}{2n^2} = \frac{1}{2}$$

证明:

- 不可满足情形: 显然, 算法无法找到可满足赋值, 因此正确宣布不可满足。
 - 可满足情形: 算法有可能找不到可满足赋值而出错, 下面证出错概率为 $\frac{1}{2^m}$ 。
1. 汉明距离定义: 设 σ^* 为最优赋值, 当前赋值 σ 与 σ^* 的汉明距离 $d(\sigma, \sigma^*)$ 表示两者取值不同的变量个数。
 2. 改进概率: 对每个不满足子句, 至少以 $\frac{1}{2}$ 概率减少 $d(\sigma, \sigma^*)$ (因子句最多 2 个文字)。
 3. 收敛分析: 经过 $2n^2m$ 步迭代, 未到达 σ^* 的概率:

$$P(\text{失败}) \leq \left(1 - \frac{1}{2n}\right)^{2n^2m} \approx e^{-nm} \leq \frac{1}{2^m}$$

1.5 本章作业

1.5.1 优惠券收集

有 n 种优惠券, 每种优惠券数目不限。每次实验随机抽取一张优惠券, 优惠券来自 n 种中任何一种的概率均等。计算需要多少次实验才能使得收集的优惠券包含 n 种中每一种至少一张。

设 X_i 表示在已收集 $i-1$ 种优惠券后, 首次抽到第 i 种新优惠券所需的次数 ($i = 1, 2, \dots, n$)。

在已收集 k 种优惠券时, 抽到新优惠券的概率为 $p_k = \frac{n-k}{n}$, 因此 X_{k+1} 服从参数为 p_k 的几何分布, 其期望为:

$$E(X_{k+1}) = \frac{1}{p_k} = \frac{n}{n-k}$$

集齐所有 n 种优惠券的总次数 T 为各阶段次数之和:

$$T = X_1 + X_2 + \dots + X_n$$

总期望为:

$$E(T) = \sum_{k=0}^{n-1} \frac{n}{n-k} = n \sum_{k=1}^n \frac{1}{k} = nH_n$$

当 n 较大时, $H_n \approx \ln n + \gamma$ 。因此:

$$E(T) \approx n \ln n + \gamma n$$

集齐 n 种优惠券的平均实验次数为:

$$E(T) = nH_n = n \left(1 + \frac{1}{2} + \dots + \frac{1}{n}\right)$$

对于大 n , 可近似为 $n \ln n + \gamma n$, 其中 $\gamma \approx 0.577$,

1.5.2 用公平硬币随机序列生成

假定你有一枚公平硬币，请设计一个有效的随机算法用来生成整数 $1, 2, \dots, n$ 的一个随机排列。并分析你的算法的时间复杂性。

Input: 整数 n (排列长度)
Output: 数组 $A[1..n]$ 包含 1 到 n 的随机排列

```

1 初始化数组  $A$ :  $A[i] \leftarrow i$  对  $i = 1$  到  $n$ ;
2 for  $i \leftarrow n$  downto 2 do
3   生成  $\lceil \log_2 i \rceil$  次公平硬币投掷，其中正面朝上为 1，反面为 0 来得到随机数的二进制串，得到
   随机数  $j \in [0, 2^{\lceil \log_2 i \rceil} - 1]$ ;
4   while  $j \geq i$  do
5     重新生成  $j$ ; // 拒绝采样保证均匀性
6   end
7   交换  $A[i]$  与  $A[j + 1]$ ;
8 end
9 return  $A$ 
```

Algorithm 11: Fisher-Yates 随机排列算法 (基于公平硬币)

考虑 i 次枚举后才找到

$$\sum_{i=1}^{\infty} q^i = \frac{q}{1-q} = 1-p \quad q = 1 - \frac{1}{2-p}$$

在最优情况下，所有的 j 都被接受，最优时间复杂度是

$$\sum_{i=2}^n \lceil \log_2 i \rceil \triangleq C_{opt}(n)$$

有：

$$C_{opt}(n) \leq \sum_{i=2}^n \log_2 n = O(n \log n)$$

也有：(此处省略分部积分的步骤)

$$C_{opt}(n) \geq \sum_{i=2}^n \log_2 i = \sum_{i=2}^n \int_{i-1}^i \log_2 x dx \geq \sum_{i=2}^n \int_{i-1}^i \log_2 x dx = \int_1^n \log_2 x dx = O(n \log n)$$

因此由夹逼定理有： $C_{opt}(n) = O(n \log n)$

下面考虑最坏情况 $C_{wor}(n)$

对于 i 情况下单次 j 的取值，拒绝概率为：

$$P_{reject}(i) = \frac{2^{\lceil \log_2 i \rceil} - i}{2^{\lceil \log_2 i \rceil}} = 1 - \frac{i}{2^{\lceil \log_2 i \rceil}} \triangleq p_i$$

考虑 i 情况下 k 次 j 的取值被拒绝的概率是 p_i^k ，每次被拒绝都会多进行 $\lceil \log_2 i \rceil$ 次硬币投掷实验，因此 k 次取值后产生的次数为： $p_i^k \cdot k \lceil \log_2 i \rceil$ ，因此 i 情况下的总投掷次数为：(跳过等差乘等比级数的数学推导)

$$w_i = \sum_{k=1}^{\infty} p_i^k \cdot k \lceil \log_2 i \rceil = \frac{p_i(1+p_i)}{(1-p_i)^2} \lceil \log_2 i \rceil = \frac{(2^{\lceil \log_2 i \rceil} - i)(2^{\lceil \log_2 i \rceil + 1} - i)}{i^2} \leq \frac{i \cdot 3i}{i^2} = 3$$

因为拒绝而多出的总次数为：

$$C_{\text{penalty}}(n) = \sum_{i=2}^n w_i \leq 3n = O(n)$$

所以最坏时间复杂度为：

$$C_{\text{wor}}(n) = C_{\text{penalty}}(n) + C_{\text{opt}}(n) = O(n \log n)$$

综上，该算法的时间复杂度是 $O(n \log n)$ (当然可以改成如果 $j > i$ ，就 $j \% i$ ，这样也就省了这步计算了)

1.5.3 矩阵互逆

给定两个 $n \times n$ 方阵 A 和 B ，判定它们是否互逆（即 $AB = I$ ）。算法基于蒙特卡罗方法，通过随机采样验证 AB 矩阵元素的性质：

- 对角线元素应为 1（允许误差 ϵ ）
- 非对角线元素应为 0（允许误差 ϵ ）

```
Input: 矩阵  $A, B$ , 误差阈值  $\epsilon$ , 采样次数  $T$ 
Output: YES 或 NO
1 for  $t \leftarrow 1$  to  $T$  do
2    $J \leftarrow \text{rand}(1, n)$  // 随机选行
3    $K \leftarrow \text{rand}(1, n)$  // 随机选列
4    $\text{ans} \leftarrow 0$  for  $l \leftarrow 1$  to  $n$  do
5      $\text{ans} \leftarrow \text{ans} + A[J][l] \times B[l][K]$ 
6   end
7   if ( $J = K$  且  $|\text{ans} - 1| > \epsilon$ ) 或 ( $J \neq K$  且  $|\text{ans}| > \epsilon$ ) then
8     return NO
9   end
10 end
11 return YES
```

- 时间复杂度： $O(T \cdot n)$
 - 每次采样需要 $O(n)$ 时间计算矩阵元素
 - 共进行 T 次采样
- 空间复杂度： $O(1)$ （仅需存储临时变量）

1.5.4 三元可满足问题

请设计随机游走算法求解三元可满足问题，并分析算法的期望运行时间。

Input: 3-SAT 公式 F , 最大迭代次数 T

Output: 满足 F 的赋值 (若找到), 否则返回 “无解”

```
1 随机初始化变量赋值  $\mathbf{x} = (x_1, \dots, x_n)$ ;  
2 for  $t \leftarrow 1$  to  $T$  do  
3   if  $\mathbf{x}$  满足  $F$  then  
4     return  $\mathbf{x}$ ;  
5   end  
6   随机选择一个不满足的子句  $C$ ;  
7   随机选择  $C$  中的一个变量  $x_j$ ;  
8   翻转  $x_j$  的值 ( $\text{True} \leftrightarrow \text{False}$ );  
9 end  
10 return “无解”;
```

- 递推关系 (3 文字子句):

$$h_j = \frac{2}{3}h_{j+1} + \frac{1}{3}h_{j-1} + 1$$

- 特征根:

$$r_1 = 1, r_2 = -\frac{1}{2} \implies h_j \approx 3^j$$

- 指数下界:

$$E[T] \geq 3^n - 1 = \Omega(3^n)$$

1.5.5 拉斯维加斯型与蒙特卡洛型的期望运行时间推导关系

设 A 是一个蒙特卡洛随机算法, 满足:

- 期望运行时间不超过 $T(n)$
- 返回正确解的概率为 $p(n)$
- 任何解的正确性可在 $T'(n)$ 时间内验证

我们构造拉斯维加斯算法 A' 如下:

- 运行 A (耗时 $\leq T(n)$)
- 验证解的正确性 (耗时 $\leq T'(n)$)
- 若正确则返回解, 否则重复验证

$$E[\text{时间}] \leq \frac{T(n) + T'(n)}{p(n)}$$

期望时间公式成立因为:

$$E[\text{总时间}] = \sum_{k=1}^{\infty} [(1 - p(n))^{k-1} p(n) \cdot k(T(n) + T'(n))] = \frac{T(n) + T'(n)}{p(n)}$$

该转换在保证结果正确性的前提下, 仅增加了多项式时间开销